

# ABSTRACT

---

Lumière is a full-stack web-based restaurant management system designed to provide a seamless dining experience for customers while offering powerful management tools for restaurant staff. The platform supports both dine-in and home delivery ordering through a single, elegant interface built with Flask (Python) on the backend and HTML, CSS, and JavaScript on the frontend, backed by a MySQL database.

The system enables customers to browse a dynamic menu organised by category (Starters, Main Course, Desserts, Beverages), add items to a cart, place orders by scanning a QR code at their table or by entering a delivery address, and request a bill — all without leaving their seat. Staff-facing panels include a Kitchen Display System (KDS) for real-time order tracking, a Billing Counter panel, a Delivery panel, and a full Admin panel for menu management, table QR-code generation, and booking administration. Live polling ensures all panels refresh automatically every eight seconds without a page reload, creating a near-real-time workflow.

Lumière aims to eliminate paper-based ordering, reduce errors, shorten service times, and enhance the overall guest experience through a premium digital interface inspired by fine-dining aesthetics — dark backgrounds, gold (#c8a96a) accents, and the Cormorant Garamond serif typeface.

## TABLE OF CONTENTS

| S. No | Topic                                                                                                                                   | Page No |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------|---------|
| 01    | Introduction<br>1.1 Objective<br>1.2 Feasibility Study<br>1.2.1 Existing System<br>1.2.2 Proposed System                                | 1–2     |
| 02    | System Analysis<br>2.1 System Specification<br>2.1.1 Hardware Requirements<br>2.1.2 Software Requirements<br>2.2 Tools and Technologies | 3–5     |
| 03    | Software Design<br>3.1 E-R Diagram<br>3.2 Data Flow Diagram<br>3.3 Use Case Diagram<br>3.4 Coding<br>3.5 User Interface                 | 6–29    |
| 04    | System Testing                                                                                                                          | 30–33   |
| 05    | Future Scope                                                                                                                            | 34      |
| 06    | Conclusion<br>6.1 Findings from the Project<br>6.2 Limitations                                                                          | 35      |
| 07    | Bibliography                                                                                                                            | 36      |

# 1. INTRODUCTION

---

Lumière is an innovative web-based restaurant management and ordering platform that redefines the dining experience by combining elegance with technology. The name 'Lumière' — French for 'light' — reflects the platform's mission to illuminate and simplify restaurant operations for both guests and staff.

The system allows restaurant guests to browse a rich, image-driven digital menu from their smartphone by scanning a QR code placed at their table, add dishes to a cart, provide special instructions, and place orders — all without waiting for a waiter. Home delivery customers can register, set a delivery address, and order remotely. On the staff side, the Kitchen Display System shows live orders, the Billing panel handles bill requests and payment tracking, and the Admin panel provides full control over the menu, bookings, and QR codes.

Built using Flask (Python), MySQL, and vanilla JavaScript with a premium dark-gold UI theme, Lumière is designed for real restaurant deployment with scalability, simplicity, and style at its core.

## 1.1 OBJECTIVE

- To provide customers with a seamless digital menu-browsing and ordering experience via QR code.
- To support both dine-in (table QR) and home delivery ordering modes from a single platform.
- To give kitchen staff a real-time Kitchen Display System (KDS) for managing orders.
- To automate billing, bill-request notifications, and payment tracking at the billing counter.
- To enable admins to manage menu items, categories, featured dishes, bookings, and table QR codes.
- To eliminate paper-based ordering and reduce service errors and wait times.

## 1.2 FEASIBILITY STUDY

### 1.2.1 Existing System

Traditional restaurant ordering relies on physical menus and handwritten order slips passed from waiters to the kitchen. This introduces several inefficiencies: orders can be misheard or miswritten, guests must wait for a waiter's attention to place or modify orders, and billing requires manual tallying. Larger restaurants often struggle to communicate order status between the floor and the kitchen in real time.

Existing digital solutions are either too expensive for small and mid-size restaurants, require proprietary hardware (tablets, POS terminals), or lack a cohesive delivery + dine-in workflow within a single system.

### **1.2.2 Proposed System**

Lumière proposes a lightweight, browser-based solution that requires no dedicated hardware beyond a standard smartphone or computer. Guests scan a printed QR code, which opens the menu directly in their browser — no app download required. The backend (Flask + MySQL) handles all order, menu, and booking data, while the frontend provides a polished, responsive interface that works on any device.

The system separates concerns cleanly: the customer-facing menu page, the kitchen KDS, the billing counter, the delivery panel, and the admin panel are all independent views that communicate through a shared REST-style API. Live polling every 8 seconds keeps all staff panels updated without complex WebSocket infrastructure.

## 2. SYSTEM ANALYSIS

---

### 2.1 SYSTEM SPECIFICATION

#### 2.1.1 Hardware Requirements

- Server / Host Machine: Any machine capable of running Python 3.10+
- Processor: Intel Core i3 or equivalent (i5/i7 recommended for production)
- RAM: Minimum 2 GB (4 GB recommended)
- Storage: Minimum 2 GB free disk space
- Client Device: Any smartphone, tablet, or desktop with a modern web browser
- Network: Wi-Fi or mobile data for QR-code-based guest access

#### 2.1.2 Software Requirements

- Python 3.10+
- Flask 3.x (web framework)
- mysql-connector-python (database driver)
- Werkzeug (file upload security)
- MySQL 8.0 / MariaDB (relational database)
- Modern web browser: Chrome, Firefox, Safari, Edge
- Google Fonts CDN (Cormorant Garamond, Lato)
- QRCode.js CDN (client-side QR generation)
- OpenStreetMap Nominatim API (reverse geocoding for GPS delivery address)

### 2.2 TOOLS AND TECHNOLOGIES

#### Flask (Python)

Flask is a lightweight WSGI micro-framework for Python. It provides routing, template rendering via Jinja2, request/response handling, and file upload support. In Lumière, Flask serves all HTML pages, handles form submissions, exposes JSON endpoints for AJAX polling (order placement, kitchen updates, billing, delivery), and manages image uploads for menu items.

#### MySQL / mysql-connector-python

MySQL is the relational database used to persist all application data including menu items, orders, bookings, and tables. The mysql-connector-python library provides direct connection management

without an ORM layer, giving fine-grained control over queries. An auto-migration function (`migrate_db`) adds new columns on startup to keep the schema in sync across deployments.

### **Jinja2 (HTML Templating)**

Jinja2 is Flask's built-in template engine. The customer-facing `index.html`, `admin.html`, `kitchen.html`, `billing.html`, `delivery.html`, `booking.html`, and `qr.html` are all Jinja2 templates that receive context variables (menu items, orders, bookings, tables) from Flask route handlers and render dynamic HTML on the server.

### **Vanilla JavaScript (Frontend Logic)**

All client-side interactivity — cart management, AJAX order placement, live polling, modal handling, GPS location detection, QR-code generation, and the Today's Special auto-slider — is implemented in plain JavaScript (ES5/ES6) without any frontend framework, keeping the bundle size zero and ensuring maximum browser compatibility.

### **CSS3 (Styling)**

The entire UI is styled with hand-crafted CSS3 including CSS variables, flexbox, CSS Grid, backdrop-filter blur effects, CSS animations (`fadeIn`, `pulse`, `blink`, `cardPopIn`), and responsive media queries for mobile, tablet, and desktop breakpoints. The dark (`#0b0f1a`) background with gold (`#c8a96a`) accent palette creates a consistent premium fine-dining aesthetic.

### **QRCode.js**

The `qrcodejs` CDN library is used in the Admin QR panel to generate printable QR codes entirely in the browser. Each QR encodes the URL `/?table=`, which the customer scans to open the menu pre-loaded with their table number, enabling automatic dine-in mode.

### **OpenStreetMap Nominatim**

The Geolocation API combined with the Nominatim reverse-geocoding service allows delivery customers to auto-fill their address by sharing their GPS location. The browser requests location permission, sends coordinates to Nominatim, and populates the address form fields automatically.

## 3. SOFTWARE DESIGN

---

### 3.1 E-R DIAGRAM

The database schema for Lumière consists of five core tables: items, orders, bookings, tables, and a derived bill view. The diagram below shows the key entities and their attributes.

| items       |
|-------------|
| id          |
| name        |
| price       |
| category    |
| type        |
| status      |
| image       |
| featured    |
| description |

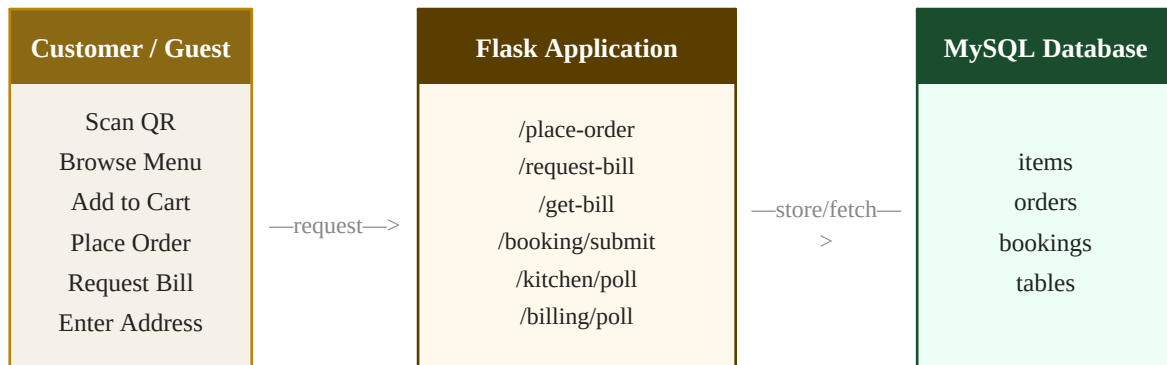
| orders           |
|------------------|
| id               |
| table_number     |
| items (JSON)     |
| total            |
| note             |
| status           |
| created_at       |
| order_type       |
| delivery_address |
| customer_name    |
| bill_sent        |

| bookings       |
|----------------|
| id             |
| name           |
| phone          |
| date           |
| time           |
| guests         |
| requests       |
| status         |
| assigned_table |
| created_at     |

| tables       |
|--------------|
| id           |
| table_number |

### 3.2 DATA FLOW DIAGRAM

The Data Flow Diagram illustrates how information moves between the Customer, the Flask application, and the MySQL database, as well as how staff panels consume that data.



### 3.3 USE CASE DIAGRAM

The system has three primary actors: Customer (Guest), Kitchen Staff, and Admin. The table below maps each actor to the use cases available to them.

| Actor           | Use Case            | Description                                                    |
|-----------------|---------------------|----------------------------------------------------------------|
| <b>Customer</b> | Browse Menu         | View categorised menu with images, prices, veg/non-veg badges  |
| <b>Customer</b> | Add to Cart         | Increment/decrement item quantities; cart persists in memory   |
| <b>Customer</b> | Place Order         | Submit cart as dine-in (table) or delivery (address) order     |
| <b>Customer</b> | Request Bill        | Send bill request to billing counter from the order panel      |
| <b>Customer</b> | Book a Table        | Fill reservation form; receive pending confirmation            |
| <b>Customer</b> | Delivery Address    | Auto-detect via GPS or enter manually; saved to localStorage   |
| <b>Kitchen</b>  | View Orders         | See pending/preparing orders split by dine-in and delivery     |
| <b>Kitchen</b>  | Update Order Status | Mark order as Preparing → Served (dine-in) or Ready (delivery) |
| <b>Billing</b>  | Send Bill           | Push itemised bill to customer's screen for dine-in tables     |
| <b>Billing</b>  | Mark Paid           | Record order as billed; update today's revenue stat            |
| <b>Delivery</b> | Mark Delivered      | Confirm delivery completed; hands off to billing for payment   |
| <b>Admin</b>    | Manage Menu         | Add / edit / delete items; toggle availability; set featured   |
| <b>Admin</b>    | Generate QR Codes   | Create per-table QR codes; print individually or all at once   |
| <b>Admin</b>    | Manage Bookings     | Confirm or reject reservations; assign table numbers           |
| <b>Admin</b>    | View All Orders     | Full order history with status controls and delete option      |

### 3.4 CODING

The following sections present the key source code modules of the Lumière platform, covering the Flask backend routes, the JavaScript cart and order logic, and selected frontend template and styling snippets.

## Backend — app.py (Flask Application)

```

from flask import Flask, render_template, request, redirect, url_for, jsonify
import mysql.connector, os, json
from werkzeug.utils import secure_filename

app = Flask(__name__)

UPLOAD_FOLDER = os.path.join('static', 'uploads')
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

def get_db():
    return mysql.connector.connect(
        host=os.getenv("DB_HOST"), user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"), database=os.getenv("DB_NAME"),
        port=int(os.getenv("DB_PORT")), ssl_disabled=False
    )

def migrate_db():
    """Auto-add new columns on startup so schema stays in sync."""
    db = get_db(); cursor = db.cursor()
    for sql in [
        "ALTER TABLE orders ADD COLUMN IF NOT EXISTS order_type VARCHAR(20) DEFAULT 'din",
        "ALTER TABLE orders ADD COLUMN IF NOT EXISTS delivery_address TEXT DEFAULT NULL",
        "ALTER TABLE orders ADD COLUMN IF NOT EXISTS customer_name VARCHAR(100) DEFAULT",
        "ALTER TABLE items ADD COLUMN IF NOT EXISTS description TEXT DEFAULT NULL",
        "ALTER TABLE orders ADD COLUMN IF NOT EXISTS bill_sent TINYINT(1) NOT NULL DEFAU",
    ]:
        cursor.execute(sql)
    db.commit(); db.close()

```

## Home Route

```
@app.route('/')
def home():
    db = get_db(); cursor = db.cursor()
    cursor.execute("SELECT * FROM items WHERE status = 'available'")
    items = cursor.fetchall()
    cursor.execute("SELECT * FROM items WHERE featured=1 AND status='available'")
    featured_items = cursor.fetchall()
    db.close()
    table = request.args.get('table', '')
```

```

        return render_template("index.html",
                                items=items, featured_items=featured_items, table=table)

```

## Place Order Route

```

@app.route('/place-order', methods=['POST'])
def place_order():
    db = get_db(); cursor = db.cursor()
    data = request.get_json()
    order_type = data.get('order_type', 'dinein')
    items = json.dumps(data.get('items', []))
    total = data.get('total', 0)
    note = data.get('note', '')

    if order_type == 'delivery':
        cursor.execute(
            """INSERT INTO orders
               (table_number, items, total, note, status,
                order_type, delivery_address, customer_name)
               VALUES (%s,%s,%s,%s,'pending','delivery',%s,%s)""",
            ('DELIVERY', items, total, note,
             data.get('delivery_address',''), data.get('customer_name','Customer'))
        )
    else:
        cursor.execute(
            """INSERT INTO orders
               (table_number, items, total, note, status, order_type)
               VALUES (%s,%s,%s,%s,'pending','dinein')""",
            (data.get('table',''), items, total, note)
        )
    db.commit()
    order_id = cursor.lastrowid
    db.close()
    return jsonify({'success': True, 'order_id': order_id})

```

## Bill Management Routes

```

@app.route('/request-bill', methods=['POST'])
def request_bill():
    table = request.get_json().get('table','')
    db = get_db(); cursor = db.cursor()
    cursor.execute(
        """UPDATE orders SET status='bill_requested'
           WHERE table_number=%s AND order_type='dinein'
           AND status IN ('pending','preparing','delivered','ready')""", (table,))
    db.commit(); db.close()
    return jsonify({'success': True})

@app.route('/billing/send/<int:id>', methods=['POST'])
def billing_send(id):

```

```

db = get_db(); cursor = db.cursor()
cursor.execute(
    "UPDATE orders SET bill_sent=1 WHERE id=%s AND order_type='dinein'", (id,))
db.commit(); db.close()
return jsonify({'success': True})

@app.route('/get-bill')
def get_bill():
    table = request.args.get('table', '')
    db = get_db(); cursor = db.cursor()
    cursor.execute(
        """SELECT id, items, total FROM orders
           WHERE table_number=%s AND order_type='dinein'
           AND bill_sent=1 AND status='bill_requested'
           ORDER BY created_at ASC""", (table,))
    rows = cursor.fetchall(); db.close()
    if not rows: return jsonify({'bill': None})
    all_items, grand = [], 0.0
    for row in rows:
        all_items.extend(json.loads(row[1]))
        grand += float(row[2])
    return jsonify({'bill': {'items': all_items, 'total': grand}})

```

## Kitchen Poll & Action Routes

```
@app.route('/kitchen/poll')
def kitchen_poll():
    db = get_db(); cursor = db.cursor()
    cursor.execute(
        "SELECT * FROM orders WHERE status IN ('pending','preparing') "
        "ORDER BY created_at ASC")
    rows = cursor.fetchall(); db.close()
    orders = []
    for o in rows:
        orders.append({
            'id': o[0], 'table': o[1],
            'items': json.loads(o[2]), 'total': float(o[3]),
            'status': o[4], 'note': o[5] or '',
            'created': str(o[6]),
            'order_type': o[7] if len(o)>7 else 'dinein',
            'delivery_address': o[8] if len(o)>8 else '',
            'customer_name': o[9] if len(o)>9 else '',
        })
    return jsonify(orders)

@app.route('/kitchen/action/<int:id>/<action>')
def kitchen_action(id, action):
    allowed = {'preparing':'preparing','delivered':'delivered','ready':'ready'}
    if action not in allowed:
        return jsonify({'error': 'invalid'}), 400
    db = get_db(); cursor = db.cursor()
    cursor.execute("UPDATE orders SET status=%s WHERE id=%s", (allowed[action],id))
    db.commit(); db.close()
    return jsonify({'success': True})
```

## Admin Menu Management Routes

```
@app.route('/admin/add', methods=['POST'])
def add_item():
    db = get_db(); cursor = db.cursor()
    name = request.form['name']; price = request.form['price']
    cat = request.form['category']; typ = request.form['type']
    desc = request.form.get('description','').strip()
    img = ''
    if 'image' in request.files:
        f = request.files['image']
        if f and f.filename and allowed_file(f.filename):
            fn = secure_filename(f.filename)
            f.save(os.path.join(app.config['UPLOAD_FOLDER'], fn))
            img = fn
    cursor.execute(
        "INSERT INTO items (name,price,category,type,status,image,featured,description)"
        " VALUES (%s,%s,%s,%s,'available',%s,0,%s)",
```

```

        (name, price, cat, typ, img, desc))
    db.commit(); db.close()
    return redirect(url_for('admin'))

@app.route('/admin/toggle/<int:id>')
def toggle_status(id):
    db = get_db(); cursor = db.cursor()
    cursor.execute("SELECT status FROM items WHERE id=%s", (id,))
    cur = cursor.fetchone()[0]
    new = 'unavailable' if cur=='available' else 'available'
    cursor.execute("UPDATE items SET status=%s WHERE id=%s", (new, id))
    db.commit(); db.close()
    return redirect(url_for('admin'))

@app.route('/admin/feature/<int:id>')
def toggle_featured(id):
    db = get_db(); cursor = db.cursor()
    cursor.execute("SELECT featured FROM items WHERE id=%s", (id,))
    cur = cursor.fetchone()[0]
    cursor.execute("UPDATE items SET featured=%s WHERE id=%s", (0 if cur==1 else 1, id))
    db.commit(); db.close()
    return redirect(url_for('admin'))

```

## Frontend JavaScript — Cart & Order Logic (script.js)

```
/* ■■ CART ■■ */
var cart = {};

function increase(name, price) {
  if (!cart[name]) cart[name] = { qty: 0, price: price };
  cart[name].qty++;
  var el = document.getElementById('qty-' + name);
  if (el) el.innerText = cart[name].qty;
  update();
}

function decrease(name) {
  if (cart[name]) {
    cart[name].qty--;
    if (cart[name].qty <= 0) {
      delete cart[name];
      var el = document.getElementById('qty-' + name);
      if (el) el.innerText = 0;
    } else {
      document.getElementById('qty-'+name).innerText = cart[name].qty;
    }
  }
  update();
}

function update() {
  var total=0, count=0, html='';
  for (var i in cart) {
    total += cart[i].qty * cart[i].price;
    count += cart[i].qty;
    html += '<div class="cart-item"><div>'+i+'</div>'
          + '<div>x'+cart[i].qty+'</div></div>';
  }
  var orders = document.getElementById('orders');
  if (orders) orders.innerHTML = html;
  document.getElementById('total').innerText = total;
  document.getElementById('cart-total').innerText = total;
  document.getElementById('cart-count').innerText = count+' Items';
  var badge = document.getElementById('navCartBadge');
  if (badge) {
    badge.textContent = count;
    badge.style.display = count > 0 ? 'flex' : 'none';
  }
}

/* ■■ PLACE ORDER ■■ */
function placeOrder() {
  if (Object.keys(cart).length === 0) {
```

```

        alert('Your cart is empty!'); return;
    }
    var isDelivery = (window.orderMode === 'delivery');
    var table = new URLSearchParams(window.location.search).get('table') || '';

    if (isDelivery && !localStorage.getItem('lumiere_user')) {
        openLoginModal('login'); return;
    }
    if (isDelivery && !localStorage.getItem('lumiere_delivery_address')) {
        openDeliveryModal(); return;
    }

    var items = Object.keys(cart).map(function(name) {
        return { name: name, qty: cart[name].qty, price: cart[name].price,
            subtotal: cart[name].qty * cart[name].price };
    });
    var total = items.reduce(function(s,i){ return s+i.subtotal; }, 0);
    var payload = { items: items, total: total,
        note: document.getElementById('orderNote').value.trim(),
        order_type: isDelivery ? 'delivery' : 'dinein' };

    if (isDelivery) {
        var addr = JSON.parse(localStorage.getItem('lumiere_delivery_address'));
        payload.delivery_address = addr.address;
        payload.customer_name = localStorage.getItem('lumiere_user') || 'Customer';
    } else {
        payload.table = table;
    }

    fetch('/place-order', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
    })
    .then(function(r){ return r.json(); })
    .then(function(data){
        if (data.success) { clearCart(); showOrderSuccess(data.order_id, isDelivery); }
        else { alert('Something went wrong. Please try again.'); }
    })
    .catch(function(){ alert('Network error. Please try again.'); });
}

```

### Bill Polling Script (injected for dine-in)

```

/* ■■ BILL POLLING (dine-in only) ■■ */
(function() {
    var table = '{{ table }}'; // injected by Jinja2
    var billShown = false;

    function pollBill() {

```

```

    if (billShown) return;
    fetch('/get-bill?table=' + encodeURIComponent(table))
    .then(function(r){ return r.json(); })
    .then(function(data){
        if (data.bill) { showBillInSummary(data.bill); billShown = true; }
    });
}

function showBillInSummary(bill) {
    var area    = document.getElementById('billDisplayArea');
    var itemsEl = document.getElementById('billDisplayItems');
    var totalEl = document.getElementById('billDisplayTotal');
    itemsEl.innerHTML = bill.items.map(function(i){
        return '<div style="display:flex;justify-content:space-between;">'
            + '<span>'+i.name+' x'+i.qty+'</span>'
            + '<span>Rs.'+i.subtotal+'</span></div>';
    }).join('');
    totalEl.textContent = 'Rs.' + bill.total;
    area.style.display = 'block';
    area.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
}
setInterval(pollBill, 6000);
pollBill();
})();

```

## CSS — Key Styling Snippets (style.css)

```
/* Fixed floating navbar */
.landing-nav {
  position: fixed; top: 16px; left: 50%;
  transform: translateX(-50%);
  width: calc(100% - 48px); max-width: 1400px;
  display: flex; align-items: center; justify-content: space-between;
  padding: 10px 28px; z-index: 999;
  background: rgba(11,15,26,0.88);
  backdrop-filter: blur(18px);
  border: 1px solid #c8a96a28; border-radius: 16px;
  box-shadow: 0 8px 32px rgba(0,0,0,0.45);
}

/* Item card */
.item {
  background: #111827; color: white; border: 1px solid #1f2937;
  border-radius: 12px; overflow: hidden;
  opacity: 0; transform: translateY(20px);
  animation: fadeIn 0.5s ease forwards;
  transition: transform .3s ease, box-shadow .3s ease;
  cursor: pointer;
}
.item:hover { transform: translateY(-5px);
              box-shadow: 0 12px 30px rgba(0,0,0,0.4); }
.item.expanded { grid-column: span 2; grid-row: span 2;
                  border-color: #c8a96a55; z-index: 10; }

/* Gold accent variables used throughout */
/* Primary gold:    #c8a96a */
/* Dark background: #0b0f1a */
/* Card bg:        #111827 */

@keyframes fadeIn { to { opacity:1; transform:translateY(0); } }

/* Hero section */
.hero {
  height: 100vh;
  background: url('l.jpeg') center/cover no-repeat;
}
.overlay {
  background: linear-gradient(160deg,
    rgba(0,0,0,0.8) 0%, rgba(0,0,0,0.55) 60%, rgba(0,0,0,0.75) 100%);
  height: 100%; display: flex; flex-direction: column;
  justify-content: center; align-items: center;
  color: white; text-align: center; padding: 0 20px;
}
.overlay h1 {
  font-family: 'Cormorant Garamond', serif;
```

```
    font-size: 90px; letter-spacing: 8px;
    line-height: 1; text-shadow: 0 4px 30px rgba(0,0,0,0.5);
}

/* Today's Special Slider */
.ts-card {
    flex-shrink: 0; width: 220px; height: 290px;
    border-radius: 14px; overflow: hidden; position: relative;
    cursor: pointer; border: 1px solid #ffffff08;
    transition: transform .32s ease, box-shadow .32s ease;
}
.ts-card:hover {
    transform: translateY(-8px) scale(1.02);
    box-shadow: 0 24px 55px rgba(0,0,0,0.7);
}
```

## Kitchen Display System — kitchen.html (Key JS)

```
function buildCard(o) {
  var isDelivery = o.order_type === 'delivery';
  var isNew      = o.status === 'pending';
  var html = '<div class="order-card '+o.status
    + (isDelivery ? ' delivery-order' : '') + '">';
  html += '<div class="order-card-header">'
    + '<span class="order-id">#'+o.id+'</span>'
    + (isNew      ? '<span class="new-badge">NEW</span>' : '')
    + (isDelivery ? '<span class="delivery-badge">DELIVERY</span>' : '')
    + '</div>';
  // Item rows
  o.items.forEach(function(item) {
    html += '<div class="item-row"><span>'+item.name+'</span>'
      + '<span class="item-qty">x'+item.qty+'</span></div>';
  });
  // Action buttons
  if (o.status === 'pending') {
    html += '<button onclick="doAction('+o.id+',\'preparing\',this)">'
      + 'Start Preparing</button>';
  }
  if (o.status === 'preparing') {
    html += isDelivery
      ? '<button onclick="doAction('+o.id+',\'ready\',this)">'
      + 'Ready for Pickup</button>'
      : '<button onclick="doAction('+o.id+',\'delivered\',this)">'
      + 'Served</button>';
  }
  html += '</div>';
  return html;
}

// Auto-refresh every 8 seconds
loadOrders();
setInterval(loadOrders, 8000);
```

## Jinja2 Template — Menu Rendering (index.html excerpt)

```
<!-- Today's Special Slider (Jinja2 loop) -->
{% if featured_items %}
<section class="ts-section" id="todays-special">
  <div class="ts-header">
    <h2 class="ts-title">Chef's <span>Picks</span></h2>
  </div>
  <div class="ts-viewport">
    <div class="ts-cards-track" id="tsCardsTrack">
      {% for item in featured_items %}
        <div class="ts-card">
          {% if item[6] %}
```

```

        
    {% else %}
    <div class="ts-card-placeholder">Dish</div>
    {% endif %}
    <div class="ts-card-info">
        <div class="ts-card-name">{{ item[1] }}</div>
        <span class="ts-card-price">Rs.{{ item[2] }}</span>
    </div>
    <button class="ts-card-add"
        onclick="increase('{{ item[1] }}',{{ item[2] }});
            event.stopPropagation()">+</button>
    </div>
    {% endfor %}
</div>
</div>
</section>
{% endif %}

<!-- Menu section – starters (veg) -->
{% for item in items %}{% if item[3]=='starter' and item[4]=='veg' %}
<div class="item" data-category="starter" data-type="veg"
    onclick="toggleExpand(this)">
    <h3>{{ item[1] }}</h3>
    <p class="item-price">Rs.{{ item[2] }}</p>
    <div class="controls">
        <button onclick="decrease('{{ item[1] }}')">-</button>
        <span id="qty-{{ item[1] }}">0</span>
        <button onclick="increase('{{ item[1] }}',{{ item[2] }})">+</button>
    </div>
</div>
{% endif %}{% endfor %}

```

## 3.5 USER INTERFACE

Lumière features a premium dark-themed UI designed to evoke a fine-dining ambience. All screens are fully responsive across mobile, tablet, and desktop. The consistent color palette — #0b0f1a background, #c8a96a gold, #111827 card — and Cormorant Garamond / Lato typefaces create a cohesive luxury brand identity.

### Hero / Landing Page

A full-viewport hero section displays a restaurant background image with a dark gradient overlay. Centered content includes the 'Est. 2024' badge, the 'Lumière' heading in Cormorant Garamond (90px), a gold divider, the tagline 'INDULGE IN REGAL CUISINE', and two CTA buttons — 'Explore Menu' (gold fill) and 'Book a Table' (ghost). Three feature cards (Fresh Daily, Chef's Special, Premium Quality) sit at the bottom of the hero.

### Fixed Navbar

A frosted-glass floating navbar (backdrop-filter blur, 88% opacity dark background, gold border) contains: the Lumière logo (SVG candlestick icon + serif wordmark), a table badge (shown when ?table=N is in the URL), a Delivery button showing the saved address, a search bar, a Veg-Only toggle switch, a profile dropdown, and a cart icon with a badge counter.

### Today's Special Slider

A horizontally scrolling card carousel showcases featured menu items. Each card (220×290px) shows the dish photo, a gradient overlay, the dish name, price in gold, a Veg/Non-Veg badge, and a '+' add-to-cart button that appears on hover. Auto-slides every 3 seconds with a progress bar; manual navigation via arrow buttons and dot indicators.

### Menu Grid

Menu items are displayed in a 4-column CSS Grid (3 on tablet, 2 on mobile). Each card shows the dish image, veg/non-veg dot badge (green/red), name, price, and +/- quantity controls. Clicking a card expands it to span 2 columns × 2 rows, revealing the description and item type badge. A bottom floating 'Menu' pill opens a category drawer for filtering.

### Order Summary Panel (Desktop)

A fixed right-side panel (260px wide) shows the current cart items, running total, a special instructions textarea, Cancel All and Place Order buttons, a Request Bill button (dine-in only), and a bill display area that auto-populates when the admin sends the bill.

### Login & Delivery Address Modals

Glassmorphism modals with animated entry (scale + translateY) handle user authentication (Sign In / Register tabs) and delivery address input (GPS auto-detect + manual fields + Home/Work/Other type selector). A success overlay with card-pop animation confirms address save.

### **Kitchen Display System (kitchen.html)**

A dark (#0a0a0a) admin panel with a sticky header showing live status, navigation links, and stats (Pending, Preparing, Delivery Orders). Orders are displayed as cards in a CSS Grid, colour-coded by status (amber left border = pending, blue = preparing). Delivery orders have a blue-tinted card background and show the full delivery address.

### **Billing Counter (billing.html)**

Sections: Bill Requested (with Send Bill + Mark Paid actions), Delivery Arrived, In Progress, and Billed Today. A revenue stat updates in real time. An alert banner flashes when a bill request arrives, and a triple-tone audio cue plays via the Web Audio API.

### **Admin Panel (admin.html)**

A full-featured management interface with: Add Item form (name, price, category, type, description, image upload), an items table with Edit/Toggle/Feature/Delete actions, a bookings table with Confirm (+ table assignment) and Reject controls, and navigation to the QR code generator.

### **QR Code Generator (qr.html)**

Admins can add individual tables or bulk-generate tables 1–N. Each table renders a QRCode.js QR code encoding `/?table=`. Individual print and Print All (CSS @media print) options produce clean single-table cards suitable for lamination.

## 4. SYSTEM TESTING

---

System testing was carried out to verify that all modules of the Lumière platform function correctly, handle edge cases gracefully, and deliver the expected user experience. The following testing methods were applied:

### Unit Testing

Individual Python route functions and JavaScript cart functions were tested in isolation. Flask routes were verified using Postman for correct HTTP status codes and JSON responses. Cart functions (increase, decrease, update) were tested with browser console assertions.

### Integration Testing

End-to-end flows were tested: placing a dine-in order → it appearing in the Kitchen KDS → marking it prepared → the Billing panel showing it → sending the bill → the customer UI displaying it. The delivery flow was similarly tested from order placement through the Delivery panel to billing.

### Responsive / UI Testing

All pages were tested on Chrome DevTools device emulations for iPhone SE (375px), iPad (768px), and desktop (1440px) to verify layout, touch targets, and font sizes.

### Security Testing

All file uploads were validated server-side using Werkzeug's `secure_filename` and an extension allowlist. SQL injection was mitigated by using parameterised queries (%s placeholders) throughout `app.py`.

### Test Cases — Order Placement

| TC# | Description                    | Input                                   | Expected Result                     |
|-----|--------------------------------|-----------------------------------------|-------------------------------------|
| 01  | Dine-in order with valid table | Items in cart, ?table=3 in URL          | Order created; Kitchen KDS shows it |
| 02  | Empty cart order attempt       | No items in cart, click Place Order     | Alert: 'Your cart is empty!'        |
| 03  | Delivery order — not logged in | Delivery mode, no login                 | Login modal opens automatically     |
| 04  | Delivery order — no address    | Logged in, no saved address             | Delivery address modal opens        |
| 05  | Valid delivery order           | Logged in, address saved, items in cart | Order placed; success overlay shown |

|    |                           |                               |                            |
|----|---------------------------|-------------------------------|----------------------------|
| 06 | Special instructions note | Text in instructions textarea | Note saved in order record |
|----|---------------------------|-------------------------------|----------------------------|

## Test Cases — Kitchen Display System

| TC# | Description               | Input                            | Expected Result                               |
|-----|---------------------------|----------------------------------|-----------------------------------------------|
| 01  | New order appears in KDS  | Order placed from customer side  | Card appears with NEW badge within 8s         |
| 02  | Mark dine-in as Preparing | Click 'Start' on pending card    | Card moves to Preparing; status updated in DB |
| 03  | Mark dine-in as Served    | Click 'Served' on preparing card | Card disappears from KDS; status = delivered  |
| 04  | Mark delivery as Ready    | Click 'Ready for Pickup'         | Card moves to Delivery panel                  |
| 05  | New order audio cue       | New order while KDS is open      | Single beep via Web Audio API                 |

## Test Cases — Billing Panel

| TC# | Description            | Input                            | Expected Result                                          |
|-----|------------------------|----------------------------------|----------------------------------------------------------|
| 01  | Customer requests bill | Tap 'Request Bill' in cart panel | Order status → bill_requested; banner appears in billing |
| 02  | Send bill to table     | Admin clicks 'Send Bill'         | bill_sent=1; customer UI shows itemised bill             |
| 03  | Mark order as paid     | Admin clicks 'Mark Paid'         | Status → billed; revenue stat updates                    |
| 04  | Revenue stat           | Multiple billed orders today     | Stat shows correct sum of today's billed totals          |

## Test Cases — Admin Panel

| TC# | Description              | Input                                      | Expected Result                                     |
|-----|--------------------------|--------------------------------------------|-----------------------------------------------------|
| 01  | Add new menu item        | Fill form with name, price, category, type | Item appears in menu immediately                    |
| 02  | Upload item image        | Select valid JPG/PNG file                  | Image saved to static/uploads; shown on menu card   |
| 03  | Toggle item availability | Click 'Out of Stock'                       | Item hidden from customer menu; row dimmed in admin |
| 04  | Feature / unfeature item | Click 'Feature'                            | Item appears/disappears in Today's Special slider   |

|    |                          |                                    |                                                   |
|----|--------------------------|------------------------------------|---------------------------------------------------|
| 05 | Generate QR codes (bulk) | Enter count=10, click Generate All | 10 QR codes rendered; printable                   |
| 06 | Confirm table booking    | Select table, click Confirm        | Booking status → confirmed;<br>table number saved |

## 5. FUTURE SCOPE

---

### **Online Payment Integration**

Integrate payment gateways such as Razorpay or Stripe to allow customers to pay their bills digitally through the app, reducing cash handling and enabling contactless checkout for both dine-in and delivery orders.

### **Real-Time Order Tracking (WebSockets)**

Replace the 8-second polling mechanism with WebSocket connections (e.g. Flask-SocketIO) for true real-time order status updates, push notifications, and instant bill delivery to the customer's device.

### **Mobile Application**

Develop native Android and iOS apps for customers (ordering) and staff (kitchen / billing) with push notifications for order status changes and bill readiness.

### **Loyalty & Rewards Program**

Introduce a points-based loyalty system where returning customers earn points on every order, redeemable as discounts — encouraging repeat visits and brand loyalty.

### **Analytics Dashboard**

Add a management dashboard showing daily/weekly/monthly revenue charts, best-selling dishes, peak ordering hours, table turnover rates, and delivery zone heatmaps.

### **Multi-Restaurant / Multi-Branch Support**

Extend the platform to support multiple restaurant branches under a single admin account, with branch-specific menus, staff panels, and consolidated reporting.

### **AI-Powered Menu Recommendations**

Use machine learning to analyse order history and suggest personalised menu items to returning customers, increasing average order value and customer satisfaction.

## 6. CONCLUSION

---

### 6.1 FINDINGS FROM THE PROJECT

Lumière successfully demonstrates how a modern, full-stack web application can transform the traditional restaurant ordering experience. The project achieves all its primary objectives: customers can browse a live digital menu, place dine-in or delivery orders, and receive their bill without any staff intervention. Kitchen staff gain a clean, auto-refreshing display that eliminates paper tickets, and the billing counter receives real-time notifications with a full order breakdown.

Key outcomes of the project include:

- A fully functional QR-code-based dine-in ordering system with no app download required.
- A dual-mode platform supporting both dine-in and home delivery from a single codebase.
- A real-time Kitchen Display System with audio cues for new orders.
- An automated bill-request and bill-delivery workflow between customer and billing counter.
- A comprehensive Admin panel for menu, booking, and QR-code management.
- A premium, responsive UI that works seamlessly across all device sizes.

The project was built entirely with open-source technologies (Flask, MySQL, vanilla JS/CSS) with zero frontend framework dependencies, keeping the solution lightweight, fast, and easy to deploy on any Python-capable hosting environment.

### 6.2 LIMITATIONS

- Authentication is client-side only (localStorage); production deployment requires server-side session or JWT security.
- The 8-second polling interval introduces a small latency; a WebSocket solution would provide true real-time updates.
- No integrated online payment — customers currently pay at the counter after receiving their digital bill.
- The delivery address is stored only in the browser's localStorage; a proper user account backend is needed for persistence.
- The platform currently supports a single restaurant branch; multi-branch support requires significant schema changes.
- There is no role-based access control on staff panels — any user with the URL can access kitchen, billing, or admin pages.

## 7. BIBLIOGRAPHY

---

- <https://flask.palletsprojects.com> — Flask Official Documentation
- <https://dev.mysql.com/doc> — MySQL 8.0 Reference Manual
- <https://jinja.palletsprojects.com> — Jinja2 Template Engine Documentation
- <https://developer.mozilla.org> — MDN Web Docs (HTML, CSS, JavaScript)
- <https://fonts.google.com> — Google Fonts (Cormorant Garamond, Lato)
- <https://nominatim.openstreetmap.org> — OpenStreetMap Nominatim (Reverse Geocoding)
- <https://cdnjs.cloudflare.com/ajax/libs/qrcodejs> — QRCode.js Library
- <https://www.w3schools.com> — W3Schools Web Development Reference
- <https://werkzeug.palletsprojects.com> — Werkzeug Utilities Documentation